

AI Study Companion — AI Tools Usage

AI Tools Usage

Two clearly separated concerns. Section 1 is about the tools used to **build** this product (process). Section 2 is about the AI used **within** the product itself (runtime features the user interacts with). Do not conflate the two.

1. AI tools used to BUILD this product

Tool	What it was used for
Claude (architecture + prompt generation)	Drafted <code>docs/architecture.md</code> (the single source of truth: schema, RAG flow, security model, event system) and the per-phase build prompts in <code>docs/build-prompts.md</code> that drove implementation order (phases 02-19).
OpenCode (coding agent)	Implemented every phase from those prompts: scaffolding, migrations, auth, services, RAG, Tutor/Quiz/Assessment/Mastery/Growth/Recommendations, analytics, admin, observability, tests/eval, deployment prep, docs. Also debugging (e.g. closing the phase-15 <code>EMBEDDING / CONCEPT_EXTRACTION</code> logging gaps, fixing the <code>RETRIEVAL_FIXTURES</code> circular-type build error) and writing the per-phase logs in <code>docs/development-prompts/</code> . Post-phase-19 continuation tasks: Home Dashboard PRD alignment (<code>services/dashboard.service.ts</code>), Admin System Health (<code>/admin/health</code>), Tutor conversation-summary continuity (<code>005_conversation_summary.sql</code> + <code>maybeRefreshConversationSummary</code>), evaluation run tracking (<code>services/evaluation.service.ts</code> + <code>/admin/ai-evaluation</code> comparison), and this documentation synchronization.
Muse Spark (model powering this session)	The large language model behind the OpenCode agent for the later phases (observability, evaluation suite, deployment, documentation, sidebar UI rework) and the continuation tasks above: code edits, test/fixture design, verification runs, and these documentation artifacts. Report issues at https://github.com/anomalyco/opencode .

What AI did **not** do: it did not provision Supabase/Vercel/Inngest accounts, did not run the live-Vercel smoke test (manual step in phase 18), and did not record the demo video. All verification claims in `docs/evaluation.md` and the phase logs come from actually executed commands (`npm test` , `npm run eval` , `npm run build` , live Supabase queries) — outputs were recorded, not hypothesized.

2. AI used WITHIN the product itself

All runtime AI goes through the thin `AIService` abstraction (`lib/ai/AIService.ts`). Provider split (current code): chat / structured generation / evaluation default to Mercury (Inception Labs, <https://api.inceptionlabs.ai/v1> , model `mercury-2.5` , reasoning model — `max_completion_tokens` headroom, temp clamped to `[0.5,1]`, `reasoning_effort=low` for structured tasks; JSON mode via `response_format: { type: "json_object" }` + server-side schema validation) and fall back to Meta's Llama API (<https://api.llama.com/compat/v1> , model `Llama-4-Maverick-17B-128E-Instruct-FP8`) when `MERCURY_API_KEY` is unset; embeddings default to local FastEmbed (<http://localhost:8000/v1> , model `BAAI/bge-small-en-v1.5` , 384 dims, Docker in `embeddings/`) with Groq (<https://api.groq.com/openai/v1> , model `nomic-embed-text-v1.5` , 768 dims) only as an explicit `EMBEDDING_PROVIDER=groq` fallback (requires re-migrating the column), because neither Mercury nor Meta exposes an embeddings endpoint (both verified 404). `chunks.embedding VECTOR(384)` matches the local default.

Feature	AI role	Human/deterministic guardrails
Grounded Tutor (<code>TUTOR</code>)	Answers questions from retrieved chunks; returns <code>{answer, confidence, grounded, citations, followUpSuggestion}</code>	Retrieval scoped by <code>project_id + user_id</code> with <code>RELEVANCE_THRESHOLD=0.25</code> ; insufficient evidence → fixed response, no LLM call; evidence wrapped in <code><retrieved_evidence></code> as untrusted data (prompt-injection tested); conversation summary + 6-message window sent as context-only (never evidence/instructions); citations validated server-side; every call logged to <code>ai_operations</code> .
Conversation summary (<code>CONVERSATION_SUMMARY</code>)	Compresses older turns into a concise rolling summary persisted on <code>conversations.summary</code>	Structured <code>{summary, keyTopics}</code> output validated + truncated (≤ 1200 chars, no sensitive data) before persistence; refresh is best-effort and never blocks the answer; logged to <code>ai_operations</code> .
Quiz generation (<code>QUIZ_GENERATION</code>)	Generates N questions per selected concepts/difficulties/types	Concept selection is backend weighted scoring (mastery + mistakes + trend + recency/frequency); output validated, retried once on shape failure; duplicate-question check vs last quiz.
Open-ended evaluation (<code>OPEN_ENDED_EVALUATION</code>)	Grades free-text answers → <code>{score 0-100, understanding,</code>	MCQ grading is deterministic (string compare, no LLM); open-ended output validated before persisting;

Feature	AI role	Human/deterministic guardrails
	strengths, missingConcepts, reasoningQuality, feedback}	failure raises instead of writing a guess.
Concept extraction (CONCEPT_EXTRACTION)	Extracts {name, description}[] from uploaded documents	Non-fatal: failure logs and yields [] so material never sticks in PROCESSING ; upsert linked via source_material_id .
Subconcept generation (SUBCONCEPT_GENERATION)	Expands a concept into finer sub-concepts for the Concepts page	Rate-limited (10/min); output validated before persisting; failure surfaces a typed error.
Flashcard generation (FLASHCARD_GENERATION)	Builds a flip-card deck from weak concepts	Rate-limited (5/min); output validated before persisting; logged to ai_operations .
Recommendations (RECOMMENDATION)	Produces {title, action_items} from weak concepts, mistakes, mastery, goal, activity	System prompt bans generic advice; validator enforces 2-5 specific items naming real concepts; skipped when nothing is weak (mastery <60 or REQUIRES_ATTENTION).
Embeddings (EMBEDDING)	Chunk + query vectors for pgvector cosine search	match_chunks RPC filtered by project_id ; JS-cosine fallback for dev; exactly one ai_operations row per call incl. failures with request_id traceability.

Deterministic (never LLM-decided): mastery math ($new = prev \times 0.7 + evidence \times 0.3$), growth classification (IMPROVING >+5 / REQUIRES_ATTENTION <-5 / else STABLE), evaluation run-tracking comparison (services/evaluation.service.ts), ownership checks (WHERE id=\$1 AND user_id=\$2 + RLS), and all analytics aggregations.

Cost/latency visibility: every call writes ai_operations(feature, model, request_id, latency_ms, success, tokens_in/out, estimated_cost, error) ; surfaced in /admin/ai-usage and project/global analytics.